ECE 251: Computer Architecture

Week 12: Exceptions, Interrupts, and Architecture Synthesis

Prof Rob Marano

Spring 2026

1. Chapter 4 Retrospective: Architecture Evolution

- **Single-Cycle**: $CPI = 1.0$, bottlenecked incredibly by the absolute longest instruction format ($T_c \approx 800 - 1000$ps).
- **Multi-Cycle**: FSM driven, faster clock boundaries ($T_c \approx 200$ps), but inflates cycle footprint ($CPI \approx 3 - 5$) operating sequentially.
- **Pipelining**: Overlaps instructions concurrently across D-Flip-Flops to marry the rapid T_c of the Multicycle with the optimal $CPI = 1.0$ limit natively.

2. The Cost of Pipelining: Complexity and Faults

- Pipelining generates extreme throughput at the cost of structural convolution.
- **Data Dependencies**: Mandates massive Forwarding loops to bypass Register caching limits.
- **Control Penalties**: Requires hardware flushing of fetching phases on mispredicted jumps natively.
- Overlapping matrices make stopping instruction tracking incredibly complex when localized sequential constraints fail unexpectedly.

3. Overview: Handling the Unexpected

- Modern processors must systematically handle logic faults or external demands natively without permanently locking up computation.
- **Exceptions**: Synchronous electrical faults generated internally by the datapath executing instructions (e.g., Undefined Opcodes, Math Overflow).
- **Interrupts**: Asynchronous external electrical signals generated outside the datapath execution flow (e.g., Network Packets, I/O Hardware).
- Both conditions force the datapath to immediately halt instruction overlap, backup tracking logic, and jump execution explicitly to an OS handler vector.

4. The Trapping Mechanisms

- **1. Exception Program Counter (EPC)**: A dedicated 32-bit hardware register tasked strictly with capturing the Program Counter memory pointer (PC) of the exact instruction generating the localized exception fault.
- **2. Cause Register**: Records a strict numerical identity classifying exactly *why* the CPU faulted natively, letting the OS run analytical branching routines.
- **3. Vector Handler Routing**: The hardware forces the multiplexer riding the PC tracking pin to dynamically rewire directly to the OS boundary at `0x8000 0180`.

5. SystemVerilog Physics: EPC and Routing

```
// The EPC Latch inside the datapath
always_ff @(posedge clk) begin
  if (Exception_Flag)
    EPC <= PC_Plus4_E - 32'd4; // Capture the failing instruction
end

// The NextPC Multiplexer expansion
always_comb begin
  if (Exception_Flag) NextPC = 32'h8000_0180; // Trap to OS
  else if (PCSrcE)    NextPC = PCBranchE;     // Standard branch
  else                NextPC = PCPlus4F;      // Sequential logic
end
```

6. Pipeline Considerations: Flushing Exceptions

- The biggest challenge is mapping faults correctly across multiple overlapping instructions.
- If an add in the Execute (EX) stage mathematically overflows, the two innocent instructions trailing directly behind it inside the Fetch (IF) and Decode (ID) stages must die instantly.
- The logic strictly utilizes the exact same `StallF` and `FlushD/FlushE` pins engineered for Branch hazards, rewriting the trailing sequence overlapping buffers into inert `NOP` structural states.

7. Retrospective: The von Neumann Architecture Journey

- **Assembly (ISA)**: Human logic bounds mapped natively to binary combinations.
- **Datapath Construction**: ALU calculation grids mapping mathematically mapped alongside immense sequential registry stores.
- **Pipelining Overlaps**: Expanding Single-Cycle bottlenecks geometrically overlapping hardware to maximize processor frequency constraints whilst preserving idealized $CPI = 1.0$.
- **SystemVerilog HDL Control**: SystemVerilog formally proves and describes these complex sequential networks to synthesize logic networks matching photolithography silicon fabrication models reliably.

8. Simulating the Processor (Part 1)

- To functionally test the SystemVerilog components, we must generate a binary executable payload for the CPU's memory arrays.
- **Step 1: Compile Assembly** – Translate your MIPS instructions (`.asm`) down to machine code (`.exe`):

```
$ python3 assembler.py test_prog.asm test_prog.exe
```

- **Step 2: Synthesize Netlist** – Bundle your CPU modules and testbench into an *Icarus Verilog* executable runtime:

```
$ iverilog -g2012 -o pipelined_cpu.vvp tb_computer.sv
```

9. Simulating the Processor (Part 2)

- **Step 3: Run the Simulation** – Execute the compiled runtime. Pass your customized binary into the CPU memory via the runtime `+PROG` hook:

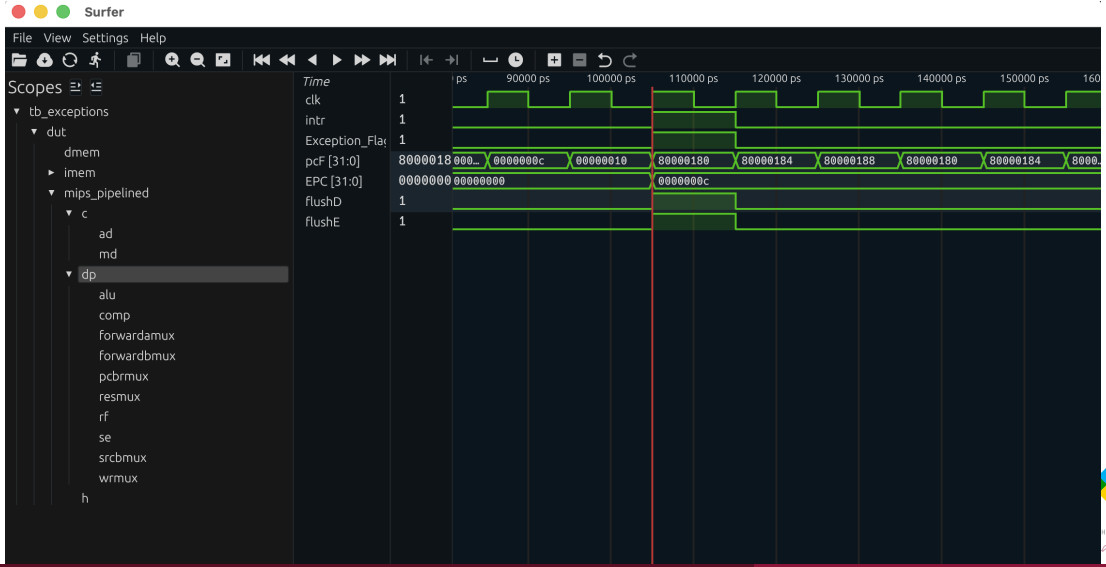
```
$ vvp pipelined_cpu.vvp +PROG=test_prog.exe
```

- The console outputs arithmetic validation metrics confirming Forwarding and Flushing mechanics operated gracefully.
- The testbench drops a digital execution wave file inside your local directory: `tb_computer.vcd`.

10. Reviewing Execution Waves with VS Code "Surfer"

- **Step 4: Inspect the Digital Waveform** – Visually inspect the pipeline delays natively utilizing the VS Code **Surfer** extension!
- Open the generated `tb_computer.vcd` waveform file directly inside Visual Studio Code.
- Track datapath variables across the timeline, heavily inspecting the `EPC`, `FlushD/E`, `StallF/D`, and dynamic ALU Forwarding boundaries.
- **Action Required:** Take meticulous screenshots of your custom Exception faulting waveforms structurally validating logic behaviors within Surfer!

11. Finding the Exception Event in the Waveform



12. Conclusion of the Processor Component

- You now possess the complete hardware logic sequence validating how generic silicon evaluates and processes procedural logic.
- Next steps: Branching beyond the processor logic specifically into Memory Hierarchies and spatial/temporal Caching optimizations spanning across RAM boundaries!